TECHNICAL PROJECT REPORT

ESP32-Based Intelligent Battery Management System

Modular Cell Monitoring, Protection, Fault-State Management and IoT Telemetry

Internship Assignment Submission
ElevanceSkills Technology Private Limited

Submitted by: Parth Khairnar
Domain: Embedded Systems / IoT
Project platform: ESP32 firmware simulation using Wokwi and Blynk
Date: 13 October 2026

Abstract

This project develops a modular, ESP32-oriented Battery Management System (BMS) engine for monitoring multiple simulated battery cells and coordinating protection, display, fault handling, and IoT telemetry. The current demonstration uses four potentiometer inputs as simulated cell-voltage sources, an I2C 16×2 LCD, relay/indicator outputs, and a Blynk dashboard. The firmware analyses cell voltages, identifies the weakest and strongest cells, calculates voltage imbalance, and supports a structured operating-state model. Non-blocking timing is used for periodic activities so that display refresh, protection handling, and communication work can progress without relying on long blocking delays. The Wokwi simulation and repository screenshots provide demonstration evidence for normal operation, cell imbalance, failsafe detection, recovery, LCD output, and Blynk dashboard views.

1. Project Overview

The objective is to build a reusable BMS software engine that can be extended beyond a four-cell demonstration. The design separates cell analysis from protection, user-interface, state-management, and telemetry responsibilities. This modular approach reduces duplicated logic and gives each subsystem a defined role. The project is a simulation/prototype and does not constitute a certified battery protection product.

1.1 Objectives

- Monitor the voltage of each configured cell and determine the minimum and maximum cell readings.
- Calculate the pack voltage spread (maximum cell voltage minus minimum cell voltage) and observe its trend over successive samples.
- Apply threshold-based protection with hysteresis and timed state transitions to avoid unstable switching.
- Present key measurements and system state on an LCD without unnecessarily redrawing the whole display.
- Send useful telemetry to a Blynk dashboard and demonstrate system behaviour during fault and connectivity scenarios.
- Keep implementation and documentation organized in a public GitHub repository.

2. System Architecture

The prototype is organized into cooperating firmware subsystems. Cell inputs are sampled and converted to voltage estimates. The BMS analysis layer derives minimum/maximum cells and imbalance information. The protection and fault-state logic evaluates measurements and determines the operating condition. The LCD layer presents local status, while the telemetry layer sends selected data to Blynk when communication is available.

Subsystem	Input / Responsibility	Output
Cell acquisition	Four potentiometer/ADC channels in simulation	Cell-voltage estimates
BMS analysis	Array of cell measurements	Weakest/strongest cell, voltage spread, trend
Protection and relay logic	Voltage/fault conditions and timers	Relay request and recovery behaviour
Fault-state manager	Fault sources and system checks	NORMAL, DEGRADED, FAILSAFE, SHUTDOWN state
LCD engine	Measurements and system state	Rotating status pages and fault display
Telemetry / Blynk	Selected measurements and events	Dashboard values and connectivity indication

3. Hardware and Software Environment

The implementation is demonstrated as an ESP32-based simulation. The four cell inputs are represented using potentiometers rather than physical battery cells, allowing repeatable voltage changes for testing.

- Controller/simulation: ESP32 in Wokwi.
- Cell-voltage test inputs: four potentiometers connected to analog inputs.
- Local display: 16×2 I2C LCD.
- Protection indication: relay output and status LED as represented in the simulation.
- IoT platform: Blynk dashboard for live values and history/analytics views.
- Development and evidence: Arduino-style firmware, Wokwi simulation, GitHub repository and screenshots.

Simulation link: https://wokwi.com/projects/477481844956329985

Repository: https://github.com/khairnarparth291-crypto/ESP32-Intelligent-Battery-Management-System

4. Modular Battery Management Engine

4.1 Cell-voltage analysis

The analysis layer treats the cell readings as an array, rather than writing separate calculation logic for each cell. For each update, the algorithm scans the configured cells and tracks the minimum and maximum readings and their indices. This provides the weakest-cell and strongest-cell information for the display, protection logic, and telemetry.

4.2 Voltage imbalance and trend

The voltage imbalance is calculated as: $\Delta V = V_{max} - V_{min}$. Comparing the current spread with the previous valid spread allows the software to classify the imbalance as increasing, decreasing, or approximately stable, subject to the implementation's comparison tolerance.

4.3 Scalability

A compile-time cell-count constant and array-based processing make the analysis structure suitable for scaling from four to sixteen cells without duplicating per-cell analysis code. For N cells, the principal scan requires O(N) time per update and O(N) storage for the voltage array. Increasing the cell count from 4 to 16 multiplies the per-scan cell work and array storage by four, while both remain linear in N. The actual pin mapping, ADC availability, input conditioning, and board resources must also be updated for physical hardware; changing the constant alone does not create additional physical ADC channels.

4.4 Adaptive imbalance threshold

The assignment calls for an adaptive threshold related to SoC or discharge rate rather than a single fixed imbalance limit. In the final implementation, the report should describe the exact inputs and formula used by the firmware. If SoC or discharge rate is not directly measured in the simulation, any estimate or placeholder must be identified as such rather than presented as a physical measurement.

5. Non-Blocking Protection Relay and Safety

The protection workflow uses time-based/state-machine logic to allow other activities to continue while the system waits for a debounce, fault persistence, or recovery interval. Hysteresis uses separate assertion and clearing conditions so that a measurement near a limit does not cause rapid relay toggling. Recovery is delayed until the fault condition has cleared and the configured verification interval has elapsed.

5.1 Sensor-anomaly considerations

The assignment specifies frozen readings, unrealistic jumps, and out-of-range measurements. These checks should be assessed against the implemented firmware and test evidence. A frozen-value detector needs a time window and a tolerance; jump detection should consider the previous sample and a plausible rate-of-change bound. A window-based check can help distinguish sustained changes from isolated noise. The current potentiometer simulation can exercise injected changes, but it does not fully reproduce real battery measurement noise or electrical faults.

5.2 Stability under fault injection

The demonstration evidence includes fault-state behaviour and recovery. To substantiate stability under continuous fault injection, testing should record the injected condition, observed state, relay response, transition timestamps, and whether the controller continues operating without reset. Do not claim exhaustive continuous-fault testing unless those tests were actually run.

6. LCD Display Engine

The LCD presents key cell readings and system information locally. A flicker-reduction approach updates only fields whose displayed text changes, instead of clearing and redrawing the entire screen on every pass. A timer-based page scheduler can rotate through battery status, system state, and telemetry information without blocking the main loop. Critical faults should take display priority over routine pages and remain visible until the relevant clear/acknowledgement condition is met.

The refresh interval should balance responsiveness with display stability and bus activity. The report should state the actual interval configured in the submitted firmware and explain why it is suitable for the simulated battery data and fault indication.

7. Fault State Machine and Recovery

The four defined operating states are NORMAL, DEGRADED, FAILSAFE, and SHUTDOWN. Their purpose is to distinguish healthy operation, operation with a non-critical issue, protective operation requiring verification, and a condition requiring shutdown. The state machine should use an enum and explicit transition conditions rather than scattered state assignments.

State	Meaning	Typical handling
NORMAL	No active fault requiring a reduced operating mode	Normal monitoring and telemetry
DEGRADED	A non-critical issue is detected	Continue safe monitoring and report the issue
FAILSAFE	A protective fault requires safe output behaviour	Apply protection and verify recovery conditions
SHUTDOWN	A critical or persistent condition requires shutdown	Maintain safe state until the defined reset/recovery criteria are met

For each transition, structured logging should capture the timestamp, previous state, new state, fault identifier, and fault source where available. The exact source coverage—cell, relay, communication, or ADC—must match the submitted implementation. Recovery from FAILSAFE should include verification instead of an immediate return to NORMAL.

8. Event-Driven Telemetry and Blynk Dashboard

The telemetry subsystem is intended to send updates when meaningful events occur or monitored values change sufficiently, reducing unnecessary transmissions. When the connection is unavailable, a fixed-size

queue can hold events for later delivery. Once connectivity returns, queued events should be replayed in insertion order, with an explicit policy for a full queue. The Wi-Fi reconnection flow should be state-based and non-blocking, while RSSI provides an indicator of wireless signal quality.

8.1 Live dashboard fields

- Individual cell-voltage readings.
- Weakest and strongest cell identifiers/readings.
- Relay and system fault-state indicators.
- Wi-Fi/telemetry connection health and offline queue depth.
- Historical plots and status summaries where configured in Blynk.

8.2 Enterprise analytics and recommendations

The advanced dashboard requirement includes historical trends, fault history, a composite risk score, maintenance recommendations, and an executive summary. The report should document the actual risk-score factors and formula, severity mapping, and update mechanism used by the project. Dashboard screenshots can support the presentation of configured widgets, but screenshots alone do not prove that every dashboard value stays synchronized with every backend state transition; that should be verified through test cases.

9. Verification Methodology and Test Scenarios

Verification is performed through controlled simulation inputs and observation of serial/LCD/dashboard outputs. Each test should have a defined initial condition, stimulus, expected behaviour, observed result, and pass/fail outcome. The following scenarios provide a compact test matrix; retain only results that were actually executed.

Test ID	Stimulus	Expected behaviour	Evidence to capture
T-01	Balanced cell inputs	Normal status; min/max and spread displayed	LCD and normal-operation screenshot
T-02	Increase one cell difference	Weakest/strongest cell and imbalance update	Cell-imbalance screenshot
T-03	Inject a protection fault	Fault state and protective output respond	Failsafe screenshot and log
T-04	Clear fault and wait for recovery	Recovery checks run before normal operation resumes	Recovery screenshot
T-05	Change inputs repeatedly near threshold	Hysteresis/debounce prevents rapid switching	Transition log over time
T-06	Disconnect network	System continues locally; queue behaviour is observable	Wi-Fi/offline evidence
T-07	Restore network	Queued telemetry is replayed in order	Queue and dashboard evidence
T-08	Review dashboard history	Configured graphs and summaries show collected data	Blynk history/analytics screenshots

10. Simulation Results and Evidence

The repository contains screenshots representing normal operation, cell imbalance, failsafe detection, recovery to normal, LCD display output, Blynk live dashboard, Blynk history/dashboard information, and Wi-Fi indication. Insert the final screenshots in this section or retain them in the repository and reference their exact filenames. Each image caption should state the condition demonstrated and what result can be observed.

- normal-operation.png — Normal operating condition and cell readings.

- cell-imbalance.png — Cell voltage spread and imbalance response.
- failsafe-detection.png — Fault detection and failsafe response.
- recover to normal.png — Recovery path after the fault condition clears.
- lcd-display.png — Local LCD output.
- blynk-live-dashboard.png — Live Blynk dashboard overview.
- blynk-history-dashboard.png — Blynk dashboard/history view; verify that the image corresponds to the caption.
- WIFI-denoting.png — Wi-Fi/telemetry connection indication.

11. Challenges Faced and Solutions

ADC scaling and simulated cell inputs: The simulated analog values required consistent conversion to a millivolt range. The conversion was corrected and checked against the potentiometer inputs.

State and relay stability: Hysteresis, debounce, and timed state transitions are used to reduce rapid switching and make recovery controlled.

LCD readability: Updating display fields rather than clearing the whole display helps reduce flicker and preserve readable output.

Telemetry during connectivity loss: A fixed-size event queue and ordered replay policy are used to handle offline telemetry, subject to the queue-full policy implemented in firmware.

Repository presentation: The project README, assumptions document, bug log, and screenshot guide organize the implementation and evidence for review.

12. Limitations and Future Improvements

- The current demonstration uses potentiometers to simulate cell voltages; real battery cells and a calibrated, protected measurement front end are not part of this simulation.
- Physical relay feedback, sensor accuracy, electrical isolation, and hardware fault behaviour require separate hardware validation.
- Scaling to 16 cells requires validating the selected controller's ADC/pin resources and adding suitable external sensing hardware if needed.
- Risk-score and maintenance-recommendation logic should be documented with its exact formula, thresholds, and test cases.
- Offline queue capacity, overflow policy, and duplicate/replay handling should be explicitly tested under extended outages.
- The system is an educational prototype and must not be used as the sole protection mechanism for a real battery pack.

13. Conclusion

The project brings together cell monitoring, imbalance analysis, protective state handling, LCD presentation, and IoT telemetry in an ESP32-oriented simulation. Its modular organization provides a foundation for further development and supports demonstration of key embedded-systems concepts. The Wokwi project, GitHub repository, and screenshots provide accessible implementation evidence. Before final submission, the report must be checked against the exact firmware version and actual test results, especially for adaptive thresholds, sensor-anomaly detection, structured logs, offline queue replay, and advanced Blynk analytics. Claims should be retained only where supported by code or recorded evidence.

Submission Links

GitHub repository: https://github.com/khairnarparth291-crypto/ESP32-Intelligent-Battery-Management-System

Wokwi simulation: https://wokwi.com/projects/477481844956329985